

A decorative graphic on the left side of the slide, consisting of white lines and circles on a blue gradient background, resembling a circuit board or data flow diagram.

REGRESI

DATA SCIENCE

Capaian Pembelajaran

dapat menjelaskan, menyiapkan, dan mengimplementasikan model regresi dengan algoritma:

A. Regresi Linier sederhana dan variabel jamak

B. Regresi Polinomial dan Non-linier

Beserta pengukuran performansinya menggunakan Python dan Scikit-learn.

Definisi

Mampu menjelaskan regresi dan bagaimana membangun model (regresi), yaitu:

- a. menyiapkan parameter model,
 - b. menggunakan tools pemodelan,
- selanjutnya menjelaskan algoritma dan menggunakan Regresi Linier dan Non-Linier, dan performansi regresi dengan Python dan Scikit-learn.

Tujuan Pembelajaran

Peserta mempelajari pengertian, cara menyiapkan, dan cara implementasi model regresi dengan algoritma:

A. Regresi Linier sederhana dan variabel jamak

B. Regresi Polinomial dan Non-linier

Beserta pengukuran performansinya menggunakan Python dan Scikit-learn.

Referensi

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J., 2011. Scikit-learn: Machine learning in Python. the Journal of machine Learning research, 12, pp.2825-2830.
- Varoquaux, G., Buitinck, L., Louppe, G., Grisel, O., Pedregosa, F. and Mueller, A., 2015. Scikit-learn: Machine learning without learning the machinery. *GetMobile: Mobile Computing and Communications*, 19(1), pp.29-33.
- <https://scikit-learn.org/stable/index.html>

Regresi

Pengertian Regresi

x : variabel bebas

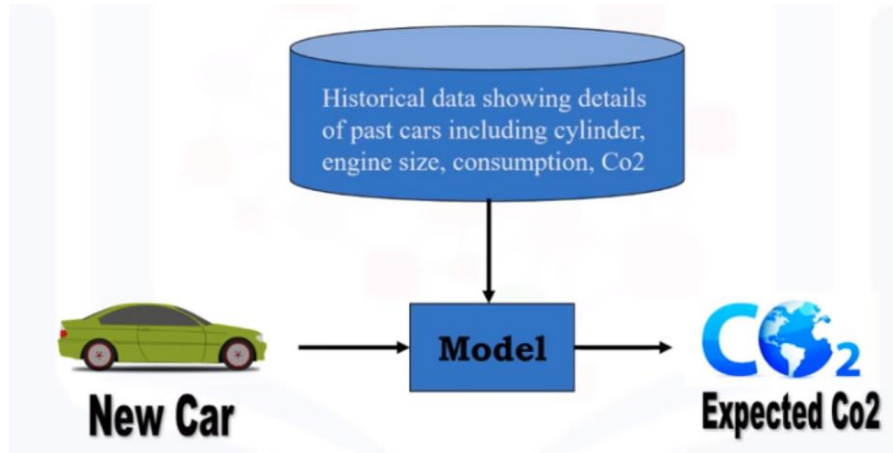
y : variabel tak bebas

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



Regresi adalah proses
Memprediksi nilai kontinu

Model Regresi



Tipe Model Regresi

Regresi Sederhana:

- Regresi sederhana linier
- Regresi sederhana non-linier
- Contoh: memprediksi co2emission vs EngineSize dari semua mobil.

Regresi Variabel Jamak:

- Regresi variabel jamak linier
- Regreasi variabel jamak non-linier
- Contoh: meprediksi co2emission vs EngineSize dan Cylinders dari semua mobil.

Aplikasi Regresi

- Prakiraan penjualan produk
- Analisis kepuasan
- Estimasi harga
- Pendapatan pekerjaan
- dst.

Algoritma Regresi

- Linier Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression
- LASSO Regression
- ANN Regression
- K-NN Regression
- dst.

Regresi Linier Sederhana

Regresi Linier Untuk Memprediksi Nilai Kontinu

x : variabel bebas

y : variabel tak bebas

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



Nilai kontinyu / numerik

Topologi Regresi Linier

Regresi Linier Sederhana:

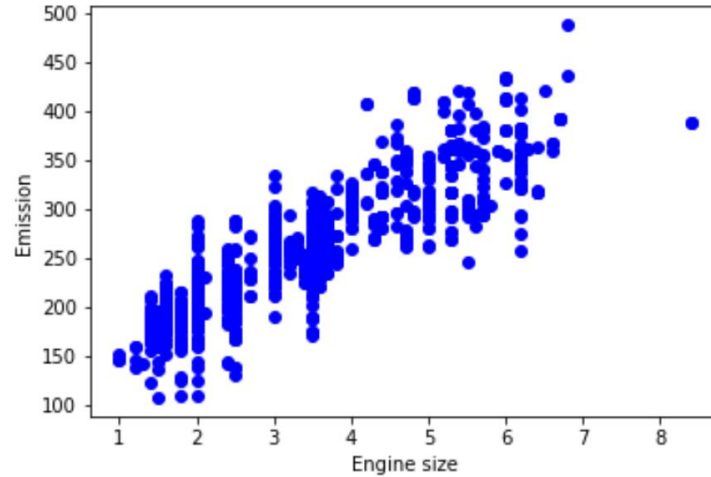
- Memprediksi co2emission vs EngineSize dari semua mobil
 - variabel bebas (x): EngineSize
 - variabel tak bebas (y): co2emission

Regresi Linier Variabel Jamak:

- Memprediksi co2emission vs EngineSize dan Cylinders dari semua mobil
 - variabel bebas (x): EngineSize, Cylinders, dst.
 - variabel tak bebas (y): co2emission

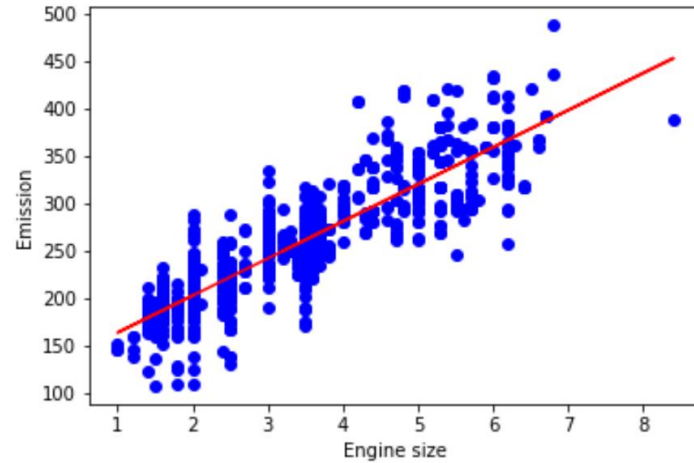
Cara Kerja Regresi Linier

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



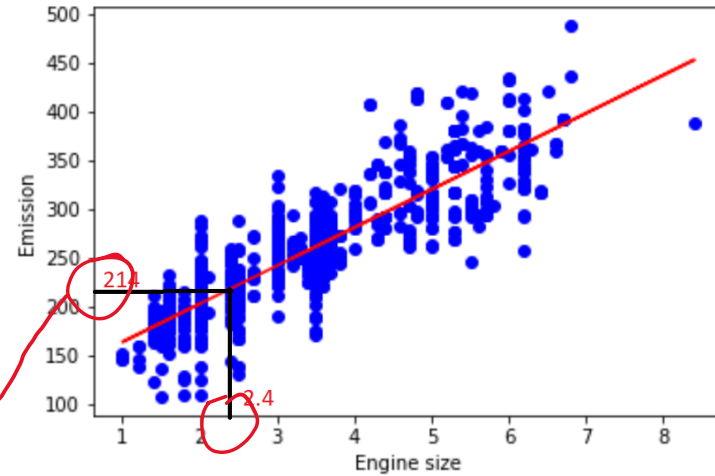
Cara Kerja Regresi Linier

	ENGINE SIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

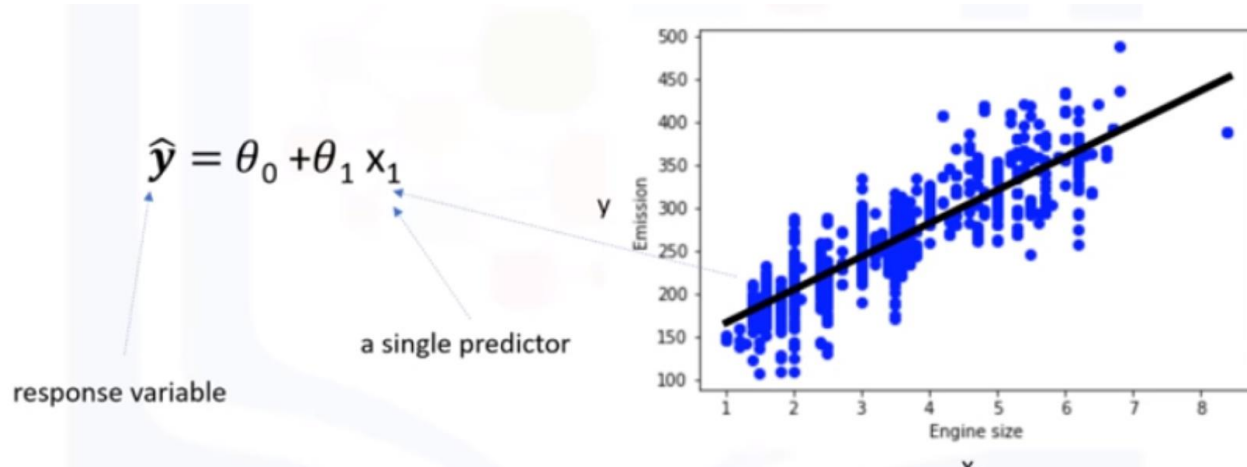


Cara Kerja Regresi Linier

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



Cara Kerja Regresi Linier



Cara Mencari Parameter Model Terbaik

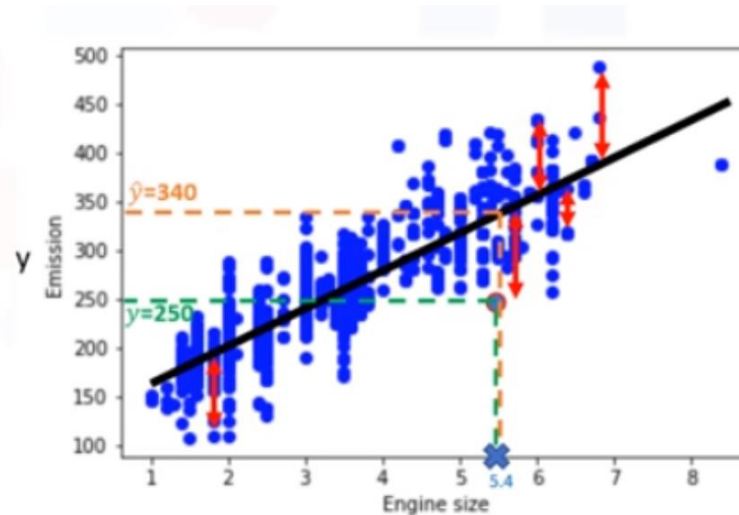
$x_1 = 5.4$ independent variable
 $y = 250$ actual Co2 emission of x_1

$$\hat{y} = \theta_0 + \theta_1 x_1$$

$\hat{y} = 340$ the predicted emission of x_1

$$\begin{aligned}\text{Error} &= y - \hat{y} \\ &= 250 - 340 \\ &= -90\end{aligned}$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Estimasi Parameter

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

$$\hat{y} = \theta_0 + \theta_1 x_1$$

$$\theta_1 = \frac{\sum_{i=1}^s (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^s (x_i - \bar{x})^2}$$

$$\bar{x} = (2.0 + 2.4 + 1.5 + \dots) / 9 = 3.34$$

$$\bar{y} = (196 + 221 + 136 + \dots) / 9 = 256$$

$$\theta_1 = \frac{(2.0 - 3.34)(196 - 256) + (2.4 - 3.34)(221 - 256) + \dots}{(2.0 - 3.34)^2 + (2.4 - 3.34)^2 + \dots}$$

$$\theta_1 = 39$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

$$\theta_0 = 256 - 39 * 3.34$$

$$\theta_0 = 125.74$$

$$\hat{y} = 125.74 + 39x_1$$

Prediksi dengan Model Regresi Linier

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

$$\hat{y} = \theta_0 + \theta_1 x_1$$

$$Co2Emission = \theta_0 + \theta_1 EngineSize$$

$$Co2Emission = 125 + 39 EngineSize$$

$$Co2Emission = 125 + 39 \times 2.4$$

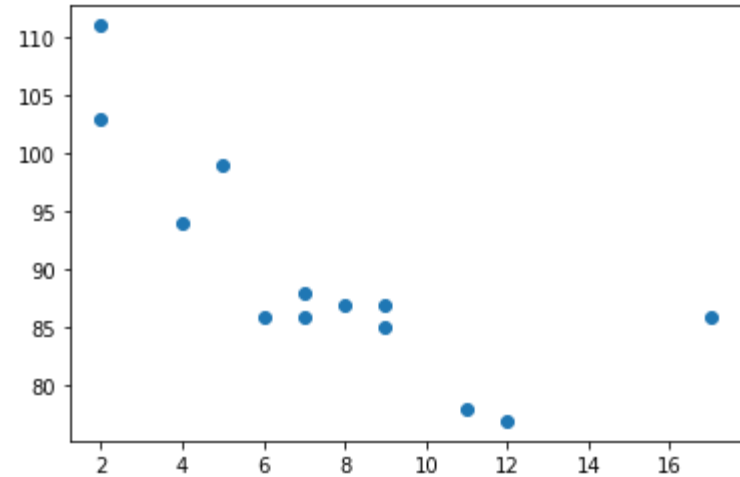
$$Co2Emission = 218.6$$

Kelebihan Regresi Linier

- Ringan
- Tidak perlu tuning parameter
- Mudah dipahami dan diinterpretasikan

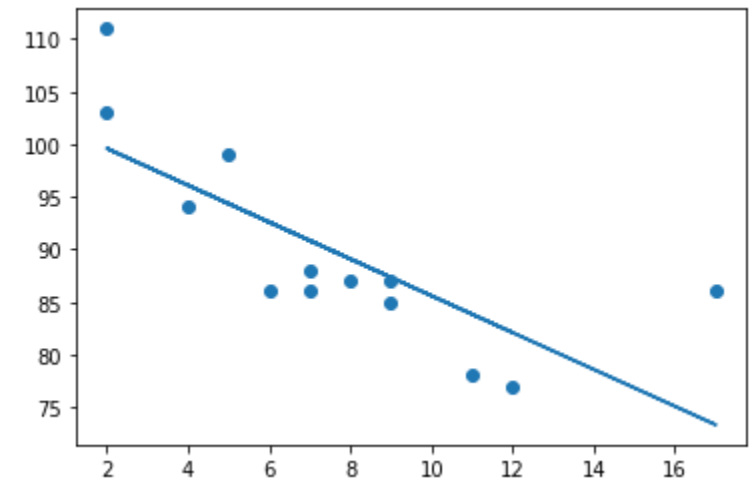
- Menggambar *scatter plot*:

```
import matplotlib.pyplot as plt
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]
plt.scatter(x, y)
plt.show()
```



- Mengimport scipy and menggambar garis dengan *Linear Regression*:

```
import matplotlib.pyplot as plt
from scipy import stats
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]
slope, intercept, r, p, std_err = stats.linregress(x, y)
def myfunc(x):
    return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```



Penjelasan

- Mengimport library yang dibutuhkan.
 - `import matplotlib.pyplot as plt`
 - `from scipy import stats`
- Membuat arrays untuk nilai-nilai pada sumbu x and y axis:
 - `x = [5,7,8,7,2,17,2,9,4,11,12,9,6]`
 - `y = [99,86,87,88,111,86,103,87,94,78,77,85,86]`
- Menjalankan method utama di *Linear Regression*:
 - `slope, intercept, r, p, std_err = stats.linregress(x, y)`
- Menggunakan fungsi untuk mendapatkan nilai kemiringan (slope dan intersep).:
 - `def myfunc(x):`
 - `return slope * x + intercept`
- Menjalankan setiap nilai dari array x menggunakan fungsi unyuk menghasilkan array baru di sumbu y. :
 - `mymodel = list(map(myfunc, x))`
- Menggambar *scatter plot* original :
 - `plt.scatter(x, y)`
- Menggambar garis *linear regression*:
 - `plt.plot(x, mymodel)`
- Menampilkan hasil diagram:
 - `plt.show()`

R untuk menampilkan Relationship

- Penting untuk mengetahui hubungan antara nilai-nilai pada sumbu-x dan nilai-nilai pada sumbu-y; jika tidak ada hubungan, regresi linear tidak dapat digunakan untuk melakukan prediksi apa pun..
- Relationship ini (koefisien *correlation*) disimbolkan dengan r .
- **Nilai r ada diantara -1 to 1, nilai 0 berarti tidak ada relationship, dan nilai 1 (dan -1) berarti 100% berelasi.**
- Python dan modul the Scipy module yang akan menghitung nilai r ini, hanya perlu memasukkan nilai x dan y ke dalamnya.
- Apakah data kita sesuai dengan *linear regression*?

```
from scipy import stats
```

```
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]
```

```
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

```
slope, intercept, r, p, std_err = stats.linregress(x, y)
```

```
print(r)
```

Memprediksi *values* sesuai dengan garis regresi

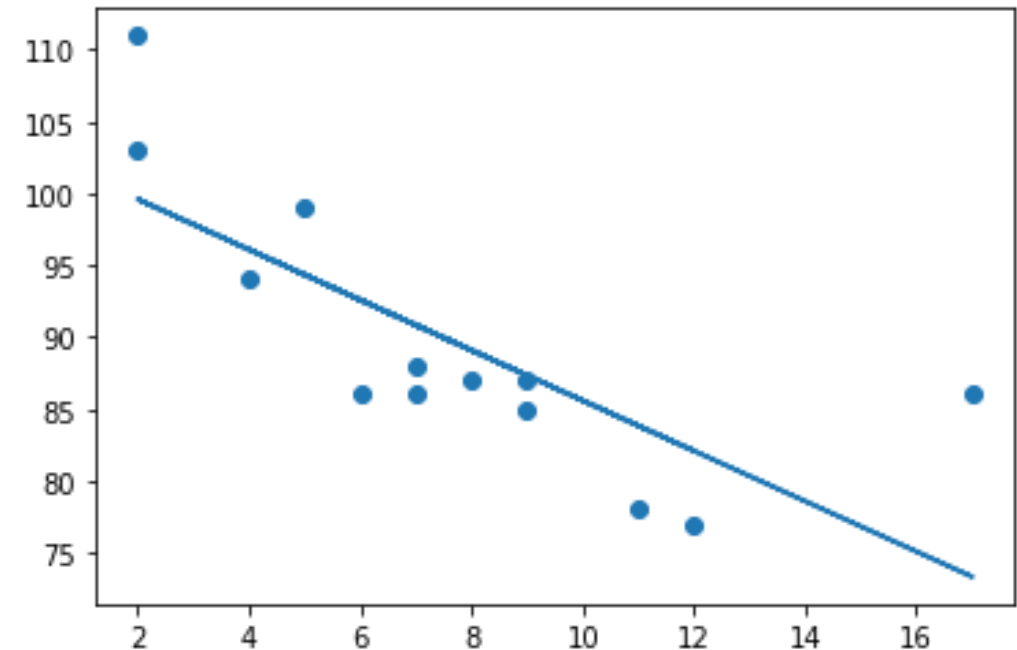
Kita dapat menggunakan informasi yang telah kita kumpulkan untuk memprediksi nilai di masa mendatang. Mari kita coba memprediksi kecepatan mobil yang berusia 10 tahun..

- Untuk melakukannya, kita memerlukan fungsi **`myfunc()`** yang sama spt contoh sebelumnya. :

```
def myfunc(x):  
    return slope * x + intercept
```

- memprediksi kecepatan mobil yg berusia 10 tahun. :

```
from scipy import stats  
x = [5,7,8,7,2,17,2,9,4,11,12,9,6]  
y = [99,86,87,88,111,86,103,87,94,78,77,85,86]  
slope, intercept, r, p, std_err = stats.linregress(x, y)  
def myfunc(x):  
    return slope * x + intercept  
speed = myfunc(10)  
print(speed)
```

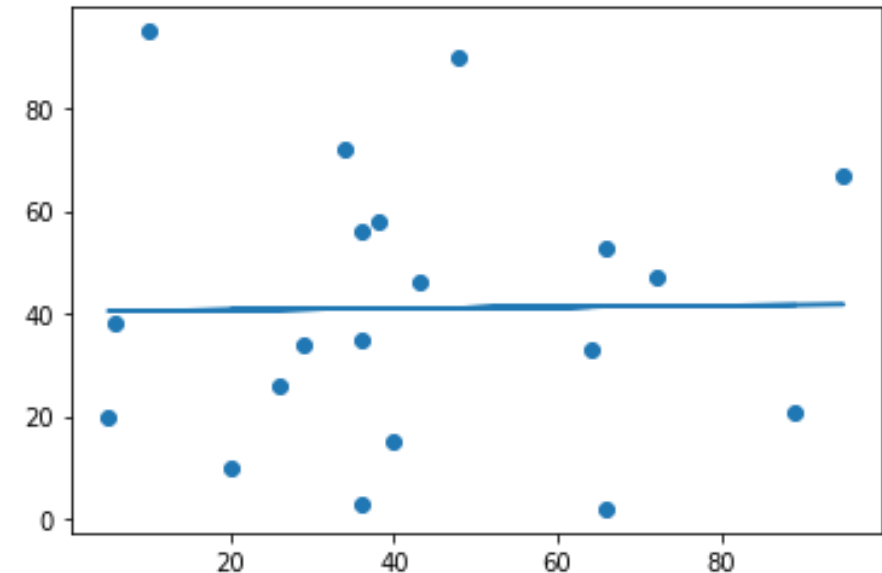


- Program tsb memprediksi kecepatan sebesar 85,6, yang juga dapat kita baca dari diagram. :

Ketepatan prediksi

- Berikut contoh di mana regresi linear bukan merupakan metode terbaik untuk memprediksi nilai di masa mendatang.
- Nilai-nilai untuk sumbu-x dan sumbu-y dibawah ini menghasilkan relasi yang sangat buruk untuk regresi linear (perhatikan sebaran titiknya) :

```
import matplotlib.pyplot as plt
from scipy import stats
x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]
slope, intercept, r, p, std_err = stats.linregress(x, y)
def myfunc(x):
    return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```



Nilai R utk relationship

- Didapatkan nilai r yang sangat rendah.

```
import numpy
```

```
from scipy import stats
```

```
x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
```

```
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]
```

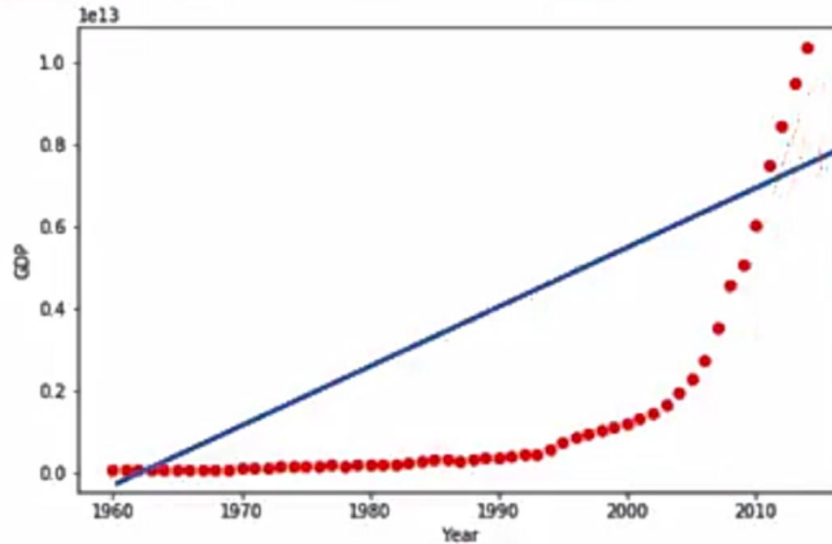
```
slope, intercept, r, p, std_err = stats.linregress(x, y)
```

```
print(r)
```

Regresi Non Linier

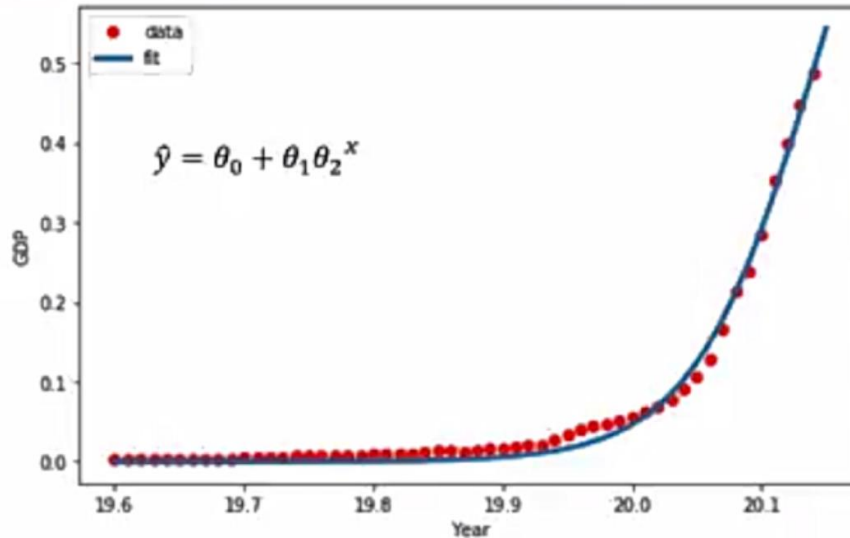
Mengapa Regresi Non-Linier Diperlukan?

	Year	Value
0	1960	5.918412e+10
1	1961	4.955705e+10
2	1962	4.668518e+10
3	1963	5.009730e+10
4	1964	5.906225e+10
5	1965	6.970915e+10
6	1966	7.587943e+10
7	1967	7.205703e+10
8	1968	6.999350e+10
9	1969	7.871882e+10
...	...	

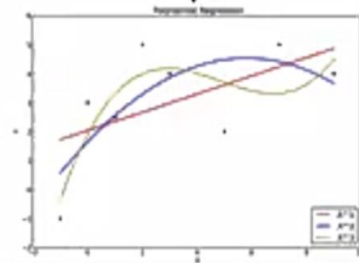
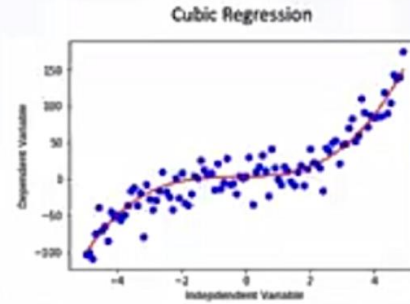
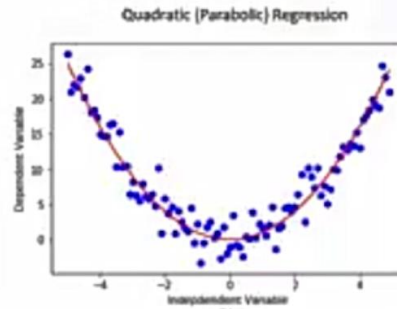
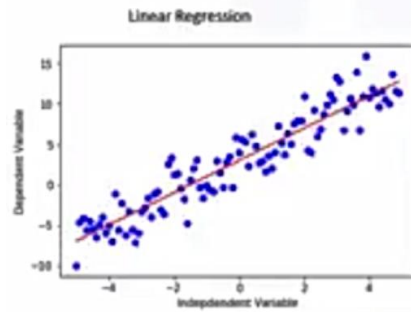


Mengapa Regresi Non-Linier Diperlukan?

	Year	Value
0	1960	5.918412e+10
1	1961	4.955705e+10
2	1962	4.668518e+10
3	1963	5.009730e+10
4	1964	5.906225e+10
5	1965	6.970915e+10
6	1966	7.587943e+10
7	1967	7.205703e+10
8	1968	6.999350e+10
9	1969	7.871882e+10
...		



Tipe Regresi



Regresi Polinomial

- Beberapa data yang berbentuk kurva dapat dimodelkan dengan regresi linier
- Contoh:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

- Model regresi polinomial dapat ditransformasikan menjadi model regresi linier.

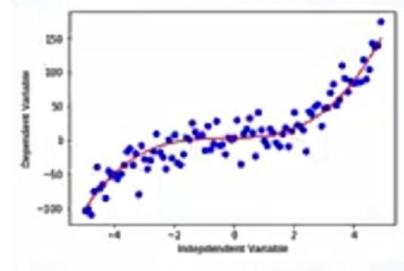
$$x_1 = x$$

$$x_2 = x^2$$

$$x_3 = x^3$$

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \rightarrow \text{dapat diselesaikan dengan least squares}$$

Regresi linier variabel jamak



Regresi Non-Linier

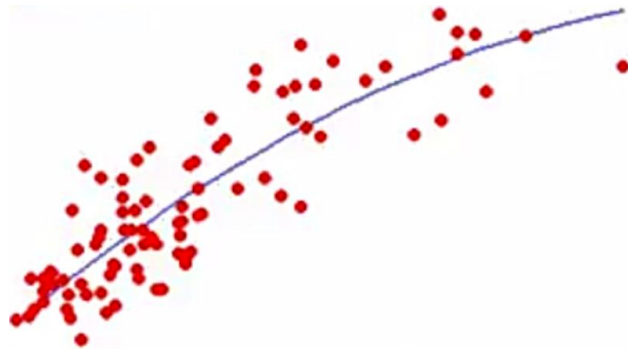
- Memodelkan hubungan tidak linier antara variabel tak bebas dengan himpunan variabel bebas
- $\hat{y}$ berupa fungsi non-linier dari parameter θ dan fitur x .

$$\hat{y} = \theta_0 + \theta_2^2 x$$

$$\hat{y} = \theta_0 + \theta_1 \theta_2^x$$

$$\hat{y} = \log(\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3)$$

$$\hat{y} = \frac{\theta_0}{1 + \theta_1^{(x - \theta_2)}}$$



Regresi Linier atau Non-Linier?

Cara untuk mengetahui apakah permasalahan cocok diselesaikan dengan regresi linier atau non linier

- Pengamatan visual atas data (visualisasi)
- Pengamatan akurasi hasil pemodelan

Cara untuk memodelkan data apabila visualisasi mengindikasikan non-linier

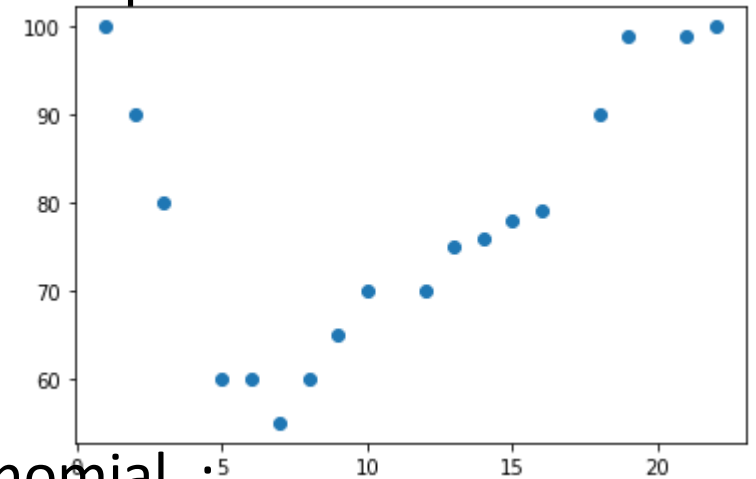
- Regresi polinomial
- Regresi non-linier
- Transformasi data non-linier menjadi linier

Polynomial Regression

- Jika titik-titik data kita tidak sesuai untuk regresi linear (garis lurus yang melewati semua titik data), maka regresi polinomial dapat menjadi pilihan yang ideal.
- Seperti halnya regresi linear, regresi polinomial memanfaatkan hubungan antara variabel x dan y untuk menentukan cara terbaik dalam membuat garis yang melewati titik-titik data tersebut.
- Python memiliki metode untuk menemukan hubungan antar-titik data serta membuat garis regresi polinomial.
- Data dibawah ini mencatat data 18 mobil yang melintasi gerbang tol.
- Data yang dicatat adalah kecepatan mobil serta waktu (jam) saat mobil melintas.

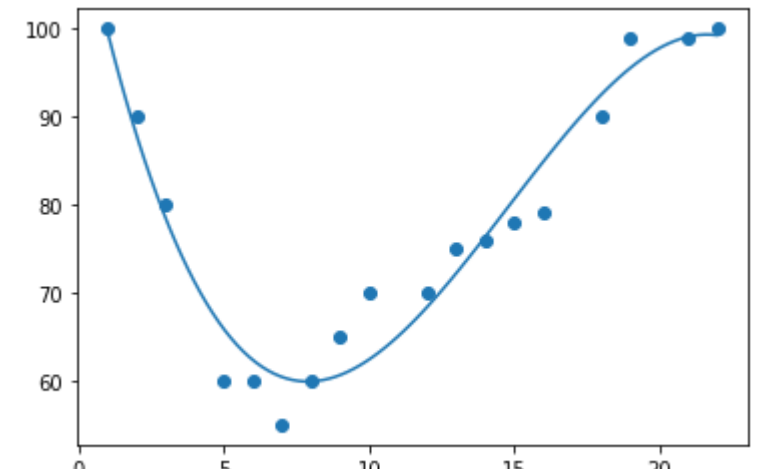
- Sumbu-x mewakili jam dalam sehari dan sumbu-y mewakili kecepatan. :
- Dimulai dengan menggambar *scatter plot*:

```
import matplotlib.pyplot as plt
x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
plt.scatter(x, y)
plt.show()
```



- Impor numpy dan matplotlib, lalu gambar garis Regresi Polinomial. :

```
import numpy
import matplotlib.pyplot as plt
x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
myline = numpy.linspace(1, 22, 100)
plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```



Penjelasan

- Import modul yang dibutuhkan.

```
import numpy
import matplotlib.pyplot as plt
```

- Buatlah array yang merepresentasikan nilai-nilai pada sumbu x dan sumbu y. :

```
x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
```

- NumPy memiliki metode yang memungkinkan kita membuat model polinomial:

```
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
```

- Kemudian, tentukan bagaimana garis tersebut akan ditampilkan; kita mulai dari posisi 1 dan berakhir di posisi 22:

```
myline = numpy.linspace(1, 22, 100)
```

- Gambar scatter plot original:

```
plt.scatter(x, y)
```

- Gambar garis *polynomial regression*:

```
plt.plot(myline, mymodel(myline))
```

- Gambar diagram:

```
plt.show()
```

R-squared

Seberapa kuat relationship antara nilai pada sumbu x dan sumbu y. Jika tidak ada hubungan, regresi polinomial tidak dapat digunakan untuk melakukan prediksi apa pun.

- Relationship diukur dengan nilai r-squared.
- **Nilai r-squared ada dalam rentang 0 sampai 1, dimana 0 berarti tidak ada relationship, dan 1 berarti 100% terrelasi.**
- Python dan modul Sklearn akan menghitung nilai ini, kita hanya perlu memasukkan array x dan y ke dalamnya. :
- Seberapa sesuai data kita dengan regresi polinomial??

```
import numpy
from sklearn.metrics import r2_score
x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
print(r2_score(y, mymodel(x)))
```

Memprediksi nilai

- Sekarang kita dapat menggunakan informasi yang telah kita kumpulkan untuk memprediksi nilai di masa mendatang.
- Sebagai contoh, kita coba memprediksi kecepatan mobil yang melewati gerbang tol sekitar pukul 17.00:
- Untuk melakukannya, kita memerlukan `*array*` ``mymodel`` yang sama seperti pada contoh sebelumnya:

```
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
```

- Prediksi kecepatan mobil yang melintas pada pukul 17.00. :

```
import numpy
```

```
from sklearn.metrics import r2_score
```

```
x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
```

```
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
```

```
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
```

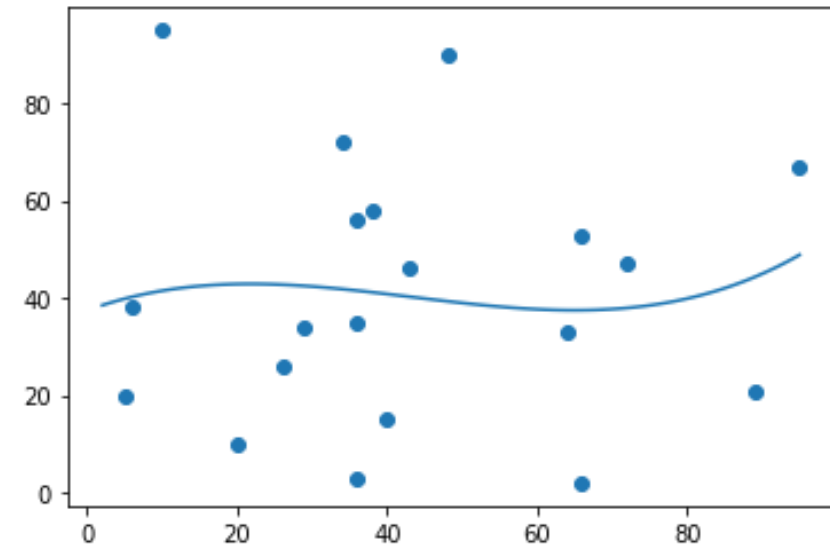
```
speed = mymodel(17)
```

```
print(speed)
```


Ketepatan memprediksi

- Contoh di mana regresi polinomial bukanlah metode terbaik untuk memprediksi nilai di masa mendatang.
- Nilai-nilai untuk sumbu-x dan sumbu-y dibawah ini menghasilkan kecocokan yang sangat buruk bagi regresi polinomial:

```
import numpy
import matplotlib.pyplot as plt
x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
myline = numpy.linspace(2, 95, 100)
plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```



Nilai R-squared

- Nilai r-squared yang sangat rendah.

```
import numpy
```

```
from sklearn.metrics import r2_score
```

```
x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
```

```
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]
```

```
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
```

```
print(r2_score(y, mymodel(x)))
```

Regresi Linier Variabel Jamak

Contoh Regresi Linier Variabel Jamak

Efektivitas variabel-variabel bebas terhadap prediksi

- Apakah kegelisahan, kehadiran dosen, dan jenis kelamin mempunyai efek pada kinerja ujian mahasiswa?

Prediksi dampak perubahan

- Seberapa besar kenaikan/penurunan tekanan darah terhadap kenaikan/penurunan BMI dari pasien?

Prediksi Nilai Kontinu pada Regresi Linier Variabel Jamak

X: Independent variable Y: Dependent variable

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

$$\text{Co2 Em} = \theta_0 + \theta_1 \text{Engine size} + \theta_2 \text{Cylinders} + \dots$$

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

$$\hat{y} = \theta^T X$$

$$\theta^T = [\theta_0, \theta_1, \theta_2, \dots] \quad X = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \dots \end{bmatrix}$$

MSE Untuk Menunjukkan Error Pada Model

$$\hat{y} = \theta^T X$$

$$\hat{y}_i = 140$$

the predicted emission of x_i

$$y_i = 196$$

actual value of x_i

$$y_i - \hat{y}_i = 196 - 140 = 56 \quad \text{residual error}$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

Estimasi Parameter Regresi Linier Variabel Jamak

Cara-cara mengestimasi parameter θ

Least Squares

- Operasi aljabar linier
- Perlu waktu yang lama untuk dataset yang besar (lebih dari 10000 baris)

Algoritma optimisasi

- Gradient Descent
- Metode yang sesuai apabila dataset sangat besar

Prediksi Menggunakan Regresi Linier Variabel Jamak

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

$$\hat{y} = \theta^T X$$

$$\theta^T = [125, 6.2, 14, \dots]$$

$$\hat{y} = 125 + 6.2x_1 + 14x_2 + \dots$$

$$Co2Em = 125 + 6.2EngSize + 14Cylinders + \dots$$

$$Co2Em = 125 + 6.2 \times 2.4 + 14 \times 4 + \dots$$

$$Co2Em = 214.1$$

Multiple Regression

- Multiple Regression/Regresi berganda mirip dengan regresi linear, namun melibatkan lebih dari satu variabel independen; artinya, kita mencoba memprediksi suatu nilai berdasarkan dua variabel atau lebih.
- Perhatikan kumpulan data di bawah ini yang memuat informasi mengenai mobil.
- Kita dapat memprediksi emisi CO2 sebuah mobil berdasarkan ukuran mesinnya, namun dengan regresi berganda, kita bisa menyertakan variabel tambahan (seperti berat mobil) untuk membuat prediksi menjadi lebih akurat.

Multiple Regression dengan Python

- Mulailah dengan mengimpor modul Pandas.
- Modul Pandas memungkinkan kita membaca file CSV dan menghasilkan objek DataFrame.
- File tersebut dapat diunduh di cyber. data.csv
`df = pandas.read_csv("data.csv")`
- Kemudian buatlah daftar nilai-nilai independen dan sebut variabel ini sebagai X.
- Masukkan nilai-nilai dependen ke dalam variabel y.
`X = df[['Weight', 'Volume']]`
`y = df['CO2']`
- Kita akan menggunakan beberapa metode dari modul sklearn, shg kita import modul tersebut:
`from sklearn import linear_model`
- Dari modul sklearn, kita akan menggunakan metode `LinearRegression()` untuk membuat objek regresi linear.
- Objek ini memiliki metode bernama `fit()` yang menerima nilai independen dan dependen sebagai parameter, serta mengisi objek regresi tersebut dengan data yang menggambarkan hubungan antarvariabel:
`regr = linear_model.LinearRegression()`
`regr.fit(X, y)`

- Sekarang kita memiliki objek regresi yang siap memprediksi nilai CO2 berdasarkan berat dan volume mobil:
- #Memprediksi emisi CO2 mobil dengan berat 2300 kg dan volume 1300 cm³:
predictedCO2 = regr.predict([[2300, 1300]])
- Contoh:

```
import pandas
from sklearn import linear_model
df = pandas.read_csv("data.csv")
X = df[['Weight', 'Volume']]
y = df['CO2']
regr = linear_model.LinearRegression()
regr.fit(X, y)
#Memprediksi emisi CO2 mobil dengan berat 2300 kg dan volume 1300 cm3:
predictedCO2 = regr.predict([[2300, 1300]])
print(predictedCO2)
```
- Result:
[107.2087328]
- Kami telah memprediksi bahwa mobil dengan mesin 1,3 liter dan bobot 2.300 kg akan melepaskan sekitar 107 gram CO2 untuk setiap kilometer perjalanan.

Coefficient

- Koefisien adalah faktor yang menggambarkan hubungan dengan variabel yang tidak diketahui.
- Contoh: jika x adalah variabel, maka $2x$ berarti x dikalikan dua. x adalah variabel yang tidak diketahui, dan angka 2 adalah koefisiennya.
- Dalam hal ini, kita dapat menanyakan nilai koefisien untuk berat terhadap CO_2 , serta untuk volume terhadap CO_2 . Jawaban yang diperoleh akan menunjukkan apa yang terjadi jika kita menaikkan atau menurunkan salah satu nilai independen tersebut.

Contoh

- Cetak nilai koefisien dari objek regresi:

```
import pandas
from sklearn import linear_model
df = pandas.read_csv("data.csv")
X = df[['Weight', 'Volume']]
y = df['CO2']
regr = linear_model.LinearRegression()
regr.fit(X, y)
print(regr.coef_)
```

- Result:

```
[0.00755095 0.00780526]
```

Explanation

- Array hasilnya mewakili nilai koefisien berat dan volume.
 - Weight: 0.00755095
 - Volume: 0.00780526
- Nilai-nilai ini menunjukkan bahwa jika berat bertambah 1 kg, emisi CO2 meningkat sebesar 0,00755095 g.
- Dan jika ukuran mesin (volume) bertambah sebesar 1 cm³, emisi CO2 meningkat sebesar 0,00780526 g.
- Hasil tebakan ini masuk akal, bisa kita uji!
- Telah kita prediksi bahwa jika sebuah mobil dengan mesin 1300 cc memiliki berat 2300 kg, emisi CO2-nya akan mencapai sekitar 107 g.
- Bagaimana jika beratnya kita tambah sebesar 1000 kg?

Contoh:

- Gunakan contoh sebelumnya, dengan mengubah bobotnya dari 2300 menjadi 3300. :

```
import pandas
from sklearn import linear_model
df = pandas.read_csv("data.csv")
X = df[['Weight', 'Volume']]
y = df['CO2']
regr = linear_model.LinearRegression()
regr.fit(X, y)
predictedCO2 = regr.predict([[3300, 1300]])
print(predictedCO2)
```

- Result:
 - [114.75968007]
- Kita telah memprediksi bahwa sebuah mobil dengan mesin 1,3 liter dan bobot 3.300 kg akan menghasilkan sekitar 115 gram CO2 untuk setiap kilometer perjalanan.
- Hal ini menunjukkan bahwa koefisien 0,00755095 tersebut benar:
 - $107.2087328 + (1000 * 0.00755095) = 114.75968$